

A Business Case for Feature-Oriented Requirements Engineering

Arnold Rudorfer, Siemens Healthcare Diagnostics

Tobias Stenzel and Gerold Herold, Siemens Healthcare

// Owing to the growing software content in medical devices, an adequate requirements engineering approach is necessary to help meet the challenges of engineering efficiency and cost effectiveness. Feature-Oriented Requirements Engineering offers such a solution. //



SYNGO, A SIEMENS business unit, is an imaging software and medical device provider with several hundred developers across multiple continents. Its projects are geographically distributed to leverage engineering talent, speed, and cost.

syngo.via software is the central hub in many healthcare workflows (see Figure 1). With it, developers can build clinical applications for computed tomography, x-ray, magnetic resonance, proton emission tomography, single-photon emission computed

tomography, oncology, and ultrasound imaging. It enables more patient exams in less time and accommodates efficient image creation, usage, archiving, and sharing for sound diagnoses. The *syngo.via* product has several thousand requirements translating into millions of C++ and C# LOC.

Like the rest of the medical device industry, Syngo faces business challenges stemming from industry regulations, which require full compliance in product development (such as traceability and auditability of requirements changes) to ensure patient safety. In addition, new market entrants, competitors from the BRIC countries (Brazil, Russia, India, and China), and IT companies moving into Siemens' traditional market space are putting pressure on cost and cycle time. Moreover, medical device functionality is increasingly being realized in software, as has happened in other industries such as the automotive sector.¹

To remain competitive amid these challenges, Syngo must continually increase productivity. Regarding software development, we hope to leverage and adapt the principles of Toyota's lean engineering approach³ to

- create customer value,
- reduce waste, and
- optimize value streams, empowerment of teams, and continuous improvement.

Toward that end, Syngo developed Feature-Oriented Requirements Engineering (FORE), a systematic approach for organizing requirements according to marketable, customer-oriented features. FORE ensures a business perspective throughout the product life cycle and is one of the enablers for product line engineering. Table 1 illustrates some ways that

FORE is helping Syngo meet requirements engineering challenges. Here, we describe FORE and the business case we developed for it.

The FORE Approach

A *feature model* is a hierarchical tree describing the structure, dependencies, commonalities, and variability of features in a product or product line (such as platforms for a medical device).^{4,5} A *feature*, in our nomenclature, is a marketable unit as an aggregation of requirements characterizing a system's capability or performance (such as a reporting feature, movie feature, or volume-rendering-technique feature). A feature is a mere container for one or more requirements. In other words, requirements specify features. The project phase dictates requirements' levels of detail (for example, considering market requirements versus software requirements).

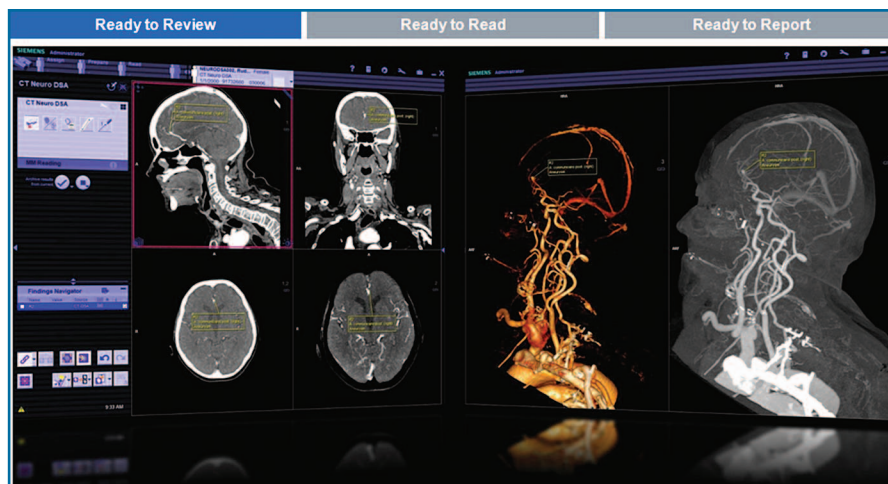


FIGURE 1. Integrated clinical workflows. Syngo.via software enables more patient exams in less time and accommodates efficient image creation, usage, archiving, and sharing for sound diagnoses.

Feature Model Benefits

Using feature models, we can structure product requirements along the customer's domain, achieving a higher

abstraction level (instead of dangling requirements) and graphical overview of the products' functionality. They also serve as a basis for proper product

TABLE 1

Requirements engineering challenges.

	Challenge	Solutions introduced by Feature-Oriented Requirements Engineering
Linkage of product requirements to business drivers	- Tedious scoping sessions with customers, owing to many specifications - Many technology-driven requirements that have no business rationale - Requirements creep or different entry points for requirements	- Setting up a business-driven feature model from the customer's viewpoint - Supporting agile methodologies by integrating the requirements database and a backlog or a project management tool - Central collection and evaluation of stakeholder requests - Early transformation of stakeholder requests into requirements
Relationship of problem to solution space	- Nontransparent relations between the problem and solution spaces - Complex product architecture, implementation, and testing - No possibility to reuse requirements and specifications for different product lines	- Modeling relations between features and components to enable impact analysis - Modeling product variability and product lines by clustering requirements to features
Very large product scope to test	- Testing without customer perspective (focusing on components) - Inefficient testing owing to a heterogeneous requirements structure	- Testing self-contained features and use cases - Continuous synchronization of up-to-date requirements with test cases
Reduce engineering overhead	A complex tracing methodology with $n:m$ relations (too many traces)	- New techniques to model cross-cutting concerns that impact various features - Simplifying tracing methods (structure-based tracing and $1:m$ relations)

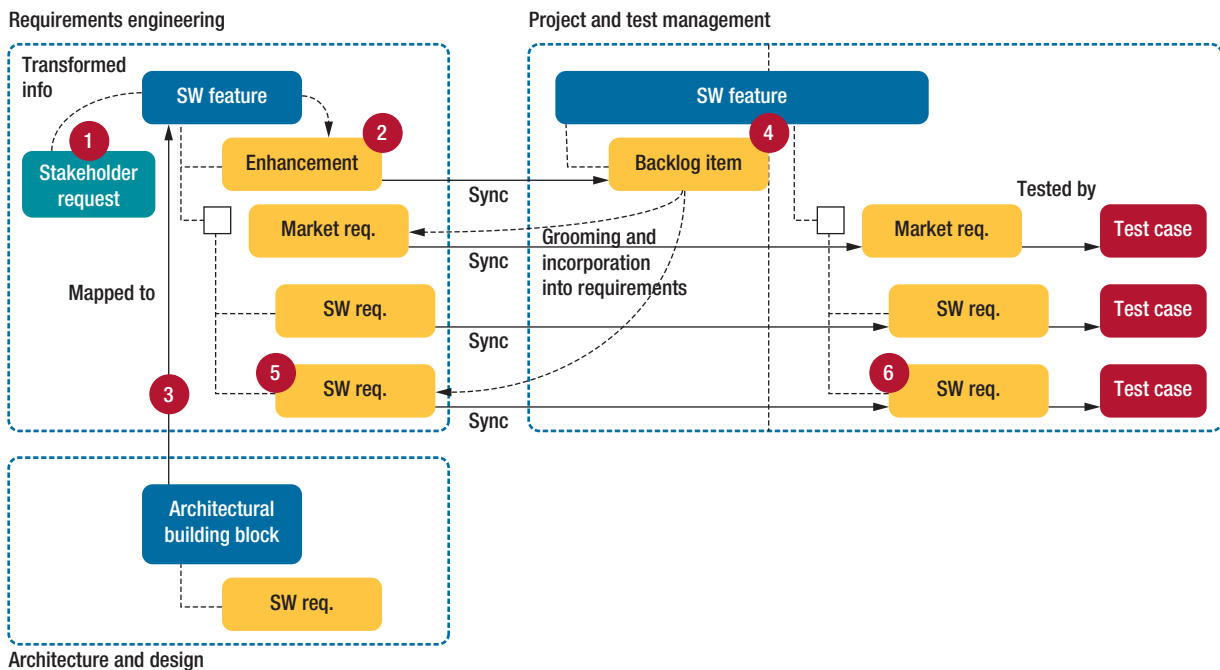


FIGURE 2. Integrating Feature-Oriented Requirements Engineering (FORE) into development. For an explanation of the steps, see the main article.

line engineering and help manage variability and commonality (for example, defining rules about whether a feature is mandatory or optional). Furthermore, developers can perfectly combine feature models with lean or agile project management methods (such as Scrum) by filling a product backlog with user stories.

Integrating Feature Models into Development

Feature models' seamless integration and application into product development processes will be key to achieving higher engineering efficiency. At Siemens Healthcare, we've integrated FORE with the engineering disciplines of architecture and design, project management, and test management.

We also introduced a tool chain supporting FORE. It consists of a tool for requirements engineering,

architecture, and design (Sparx Enterprise Architect) and a tool for backlog and test management (Microsoft Team Foundation Server).

The following development use case and steps summarize our approach (see Figure 2):

1. Product managers collect, analyze, and decide on stakeholder requests from various sources, such as end customers or internal stakeholders.
2. If a requirements engineer accepts the stakeholder requests, he or she transforms the requests into completely new software (SW) features or enhancements to existing features.
3. Together with architects, the requirements engineer maps the SW features to architectural building blocks to determine complexity and effort.

4. Then, the product manager synchronizes the enhancements per feature in parallel as items in the product backlog for prioritization and release planning.

5. As the assigned software development Scrum teams further decompose and groom the backlog items, they incorporate changes on the SW requirement and design levels.
6. Both market and SW requirements sync to the test management suite to enable tracing to test cases.

Introducing FORE and integrating it in a Scrum-based development setup has provided multiple benefits. The strict distinction of stakeholder requests and SW features and their enhancements avoids redundancies and inconsistencies in product requirements (such as a stakeholder request that might address an existing feature

or enhancement). Besides representing the formal design input of the medical device, the feature model serves as the primary input source for the product backlog and enables Scrum-based project management. Mapping architectural building blocks to SW features provides early information about complexity and input for subsequent effort estimations. Moreover, testers always get up-to-date requirements that are ready to be traced to test cases. In this regard, the feature model is the central information hub and provides a collaboration basis for various processes and disciplines.

The Business Case

Although we already had experience doing business cases for software process improvement, FORE's business case was one of the most difficult to develop. This difficulty was due to the nature of requirements engineering and the many relations the discipline has with adjoining engineering workflows. Figure 3 shows the business case approach's four major steps: assessing the type of investment, costs, benefits, and risk.

Assessing the Type of Investment

To investigate the investment's characteristics, we first determined five questions:

- What's the savings potential with FORE?
- What's FORE's return on investment (ROI)?
- By when will we recover the investment?
- Which engineering areas contribute most to the business case?
- Finally, what happens if important levers fail? (Levers are critical success factors that will help us realize FORE's benefits.)

We investigated the answers later, when assessing benefits and risk.

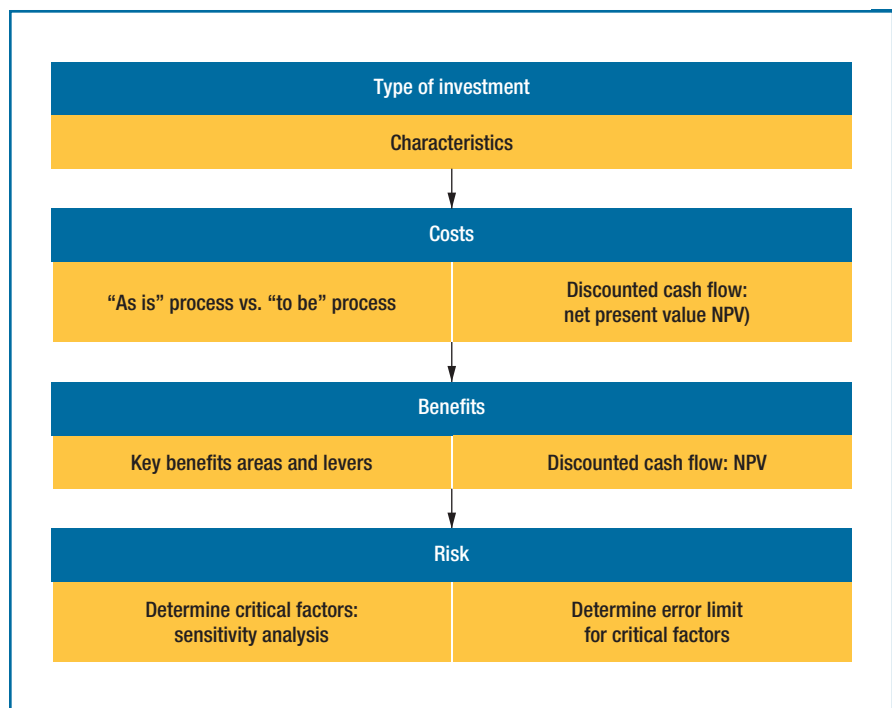


FIGURE 3. Our business case approach involved four major steps: assessing the type of investment, costs, benefits, and risk.

Assessing Costs

Next, we determined an “as is” requirements engineering process (the current state) and defined the “to be” process (the future state—that is, FORE). This way, we could accurately record the as-is state's operating costs. Moreover, we could subsequently estimate preparation and rollout costs. This included the costs for needed process changes (methodology and tooling), including staff training and organizational change management.

Assessing Benefits

To determine the key benefits and levers, we ran a series of workshops with key stakeholders from development (product managers, architects, test managers, and project managers). We identified four key benefits. The first was more effective testing and easier bug fixing. The second was transparency and easy overview of product functionality. The third was reduced product complexity

through variability modeling, and the final benefit was easier tracing owing to fewer trace relationships.

Effective testing and easier bug fixing.

One of FORE's levers is a simplified manner for creating test cases. This is possible owing to a better overview for test cases. We spend less time on understanding the requirements and their associated use cases. Additionally, there's a much earlier consideration of nonfunctional requirements such as scalability and performance. An explicit modeling attached with an integrated view lets us detect nonfunctional requirements issues early in the process. We can quickly detect any omission.

Transparency. This area relates to the requirements content. A feature model, owing to its graphical nature, provides a quick overview from a customer's perspective on how product

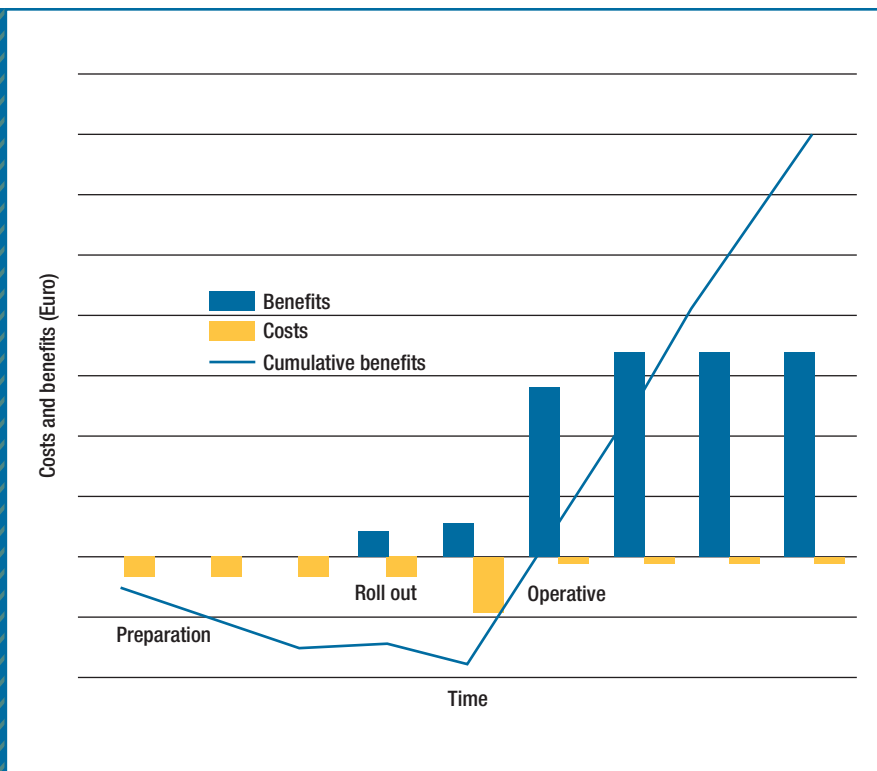


FIGURE 4. FORE benefits calculation. The net present value was approximately 15 to 20 percent of an annual R&D budget. The time of payback was approximately two years, resulting in an ROI of approximately 1:3.

requirements are structured. Furthermore, we can trace each feature from the business need to the customer requirement. A business-driven feature model has the benefits of allowing us to more easily define a product release and to more quickly understand the requirements content.

First, it's easier to prepare and conduct scoping sessions to define a product release. Given the logical decomposition, early on, we can define the boundaries for a product development release as we select the number of features that fit our resource and time constraints. Second, we can quickly bring product managers up to speed when looking at a graphical model (rather than having them parse through many text-only product requirements descriptions). In the context of our product, we needed to deal

with more than 5,000 requirements. The resulting feature model had more than 800 features. Industry practitioners report that using graphical models in requirements engineering helps reduce the time to understand and review product requirements by 40 percent.⁶ In this case, the product structure's transparency helps us determine where to zoom in—for example, when reviewing a clinical workflow.

Reduced complexity. We can reduce product complexity through product variability modeling. Applying this in our project helped reduce redundancy in the product architecture.

Simplified tracing. The feature model's tree structure mirrors the vertical traces from a business need to product requirements, design, and code. This

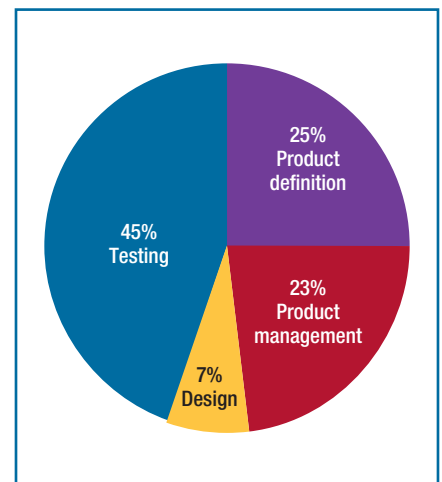


FIGURE 5. The distribution of benefits across engineering disciplines. Testing is the biggest beneficiary of applying FORE.

implicit tracing mechanism is built when we construct the feature model. So, we have fewer traces to manage and fewer opportunities to inject tracing errors when we must retrace. Therefore, rework also decreases.

Benefits analysis. Equipped with the benefits data, we can calculate the net present value (NPV) and the time of payback (see Figure 4 for an example). We used the time-savings-times-salary method to calculate benefits.⁷ The NPV in our case was approximately 15 to 20 percent of an annual R&D budget. The time of payback was approximately two years, resulting in an ROI of approximately 1:3.

Another interesting result was the distribution of the resulting benefits across the product development disciplines: product definition, project management, design, and testing (see Figure 5). We could conclude that testing would be the biggest beneficiary of applying FORE. Furthermore, to understand the total set of benefits, we had to look holistically at all product development disciplines. It was also best to continue reviewing and confirming benefits with stakeholders

from all those disciplines. We'd need those stakeholders as champions to roll out FORE and to continue reiterating investment benefits to development staff.

Assessing Risk

Finally, we determined how levers behave. Running a sensitivity analysis on the key levers with large benefits showed that the NPV was still positive. So, implementing FORE still made sense.

FORE ensures that product requirements only get into product releases that create customer value. This avoids unnecessary up-front specifications of product requirements that won't be implemented. Thus, engineering effort isn't wasted.

The feature model as a central development artifact helped our development organization optimize value streams in architecture and testing. A key lesson we learned from doing the business case is that FORE helps sustain focus in actual implementation. Furthermore, we can develop a business case only if we have completely specified FORE. Changing a key process development step requires enduring support from all levels of the organization and the involvement of key stakeholders.

We can take further steps to increase FORE's utility through automation. For example, requirements quality checking via rules in a feature model would help ensure consistency and data quality. This would also help product managers and requirements engineers catch requirements defects early. Furthermore, in product risk management, product risks could be linked to requirements in a feature node. This would help save time in finding product risk and the corresponding mitigation. 

ABOUT THE AUTHORS



ARNOLD RUDORFER is the program manager for system platform development at Siemens Healthcare Diagnostics. His research interests include requirements engineering, organizational change management, software technologies, and business strategy. Rudorfer has an MS in software engineering and business administration from the University of Technology Graz, Austria. Contact him at arnold.rudorfer@siemens.com.



TOBIAS STENZEL is a process manager for requirements engineering and product management at Siemens Healthcare. His research interests include requirements engineering- and product management-related topics. Stenzel has an MS in computer science from the University of Technology Munich, Germany. Contact him at tobias.stenzel@siemens.com.



GEROLD HEROLD is a program manager for change management projects and a process manager for project management at Siemens Healthcare. His research interests include globally distributed, large software development projects from technical and business perspectives. Herold has an MS in computer science and an MS in electrical engineering, both from the University of Erlangen-Nürnberg, Germany. Contact him at gerold.herold@siemens.com.

Acknowledgments

We thank Siemens colleagues Christa Schwaninger, Peter Hofman, Klaus Moritzen, Thomas Baer, and particularly our master's student, Felix Popp, for input, discussion, and reviews. We also extend our appreciation to the Syngo global head of R&D, Michael Heinold, for providing senior management support to make software development improvement happen.

References

1. G. Reichart and M. Haneberg, *Key Drivers for a Future System Architecture in Vehicles*, tech. report 2004-21-0025, SAE Int'l, 2004.
2. C. Smoak, "Medical Device & Diagnostic Industry 101," *Proc. PharmaSUG*, Pharmaceutical Industry SAS Users Group, paper 1B01, 2010; www.lexjansen.com/pharmasug/2010/ib/ib01.pdf.
3. T. Ohno and N. Bodek, *Toyota Production System: Beyond Large-Scale Production*, Productivity Press, 1988.
4. C. Kang and S. Cohen, *Feature-Oriented Domain Analysis (FODA) Feasibility Study*, tech. report CMU/SEI-90-TR-021, Software Eng. Inst., Carnegie Mellon Univ., 1990.
5. B. Berenbach et al., *Software Systems Requirements Engineering*, McGraw Hill, 2009.
6. B.A. Berenbach, "Comparison of UML and Text Based Requirements Engineering," *Proc. 19th Ann. ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications Conf. (OOPSLA 04)*, ACM, 2004, pp. 247–252.
7. B. Mutschler, N. Zarvic, and M. Reichert, *A Survey on Economic-Driven Evaluations of Information Technology*, tech. report, Centre for Telematics and Information Technology, Univ. Twente, 2007.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.